COMP1860 Computer Systems and Architecture

S1.W6.L1: Computer Architecture and Instruction Sets

Leandro Soares Indrusiak

Learning outcomes

- Explain basic building blocks of a computer, e.g., ALU, registers, CPU, memory.
- Explain how instructions are represented at the machine level and in the context of a symbolic assembler.

Text Books

- Noam Nisan and Shimon Schocken, **The Elements of Computing Systems: Building a Modern Computer from First Principles**
  - MIT Press, 2nd Edition, 2021
- William Stallings, **Computer Organisation and Architecture: Designing for Performance**,
  - Pearson Education, 11th edition, 2021

Drop-in Lab sessions in Week 6 to Week 11

- Mondays 14:00 - 16:00, Bragg Computing UG teaching lab (2.05)
- Wednesdays 12:00 - 14:00, EC Stoner Cluster (6.61)
- Fridays 11:00 - 13:00, Worsley LIDA Cluster (11.06)

You are welcome to any of these sessions if there is space

Overview

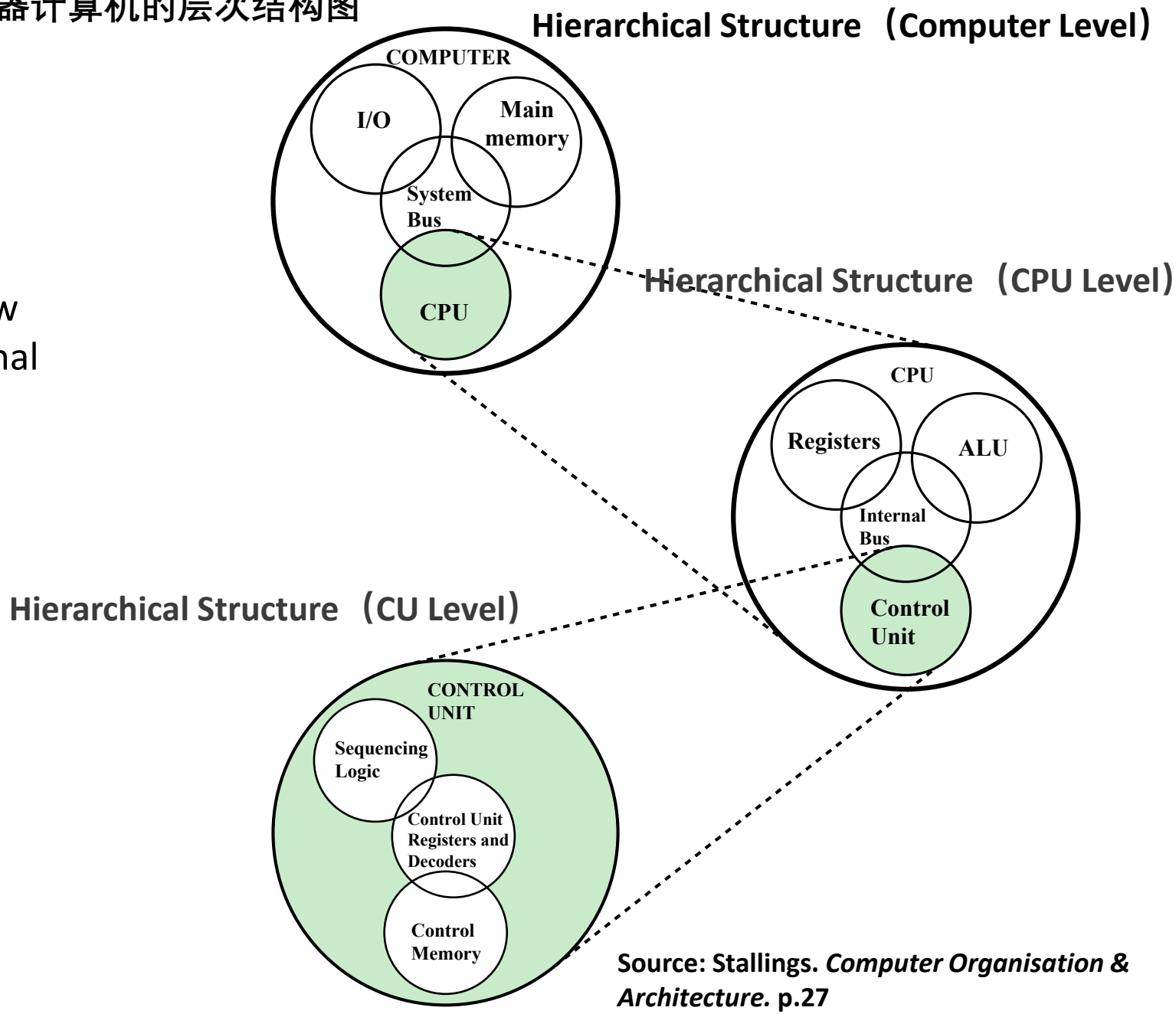
1. Part I – How a computer runs a program: components, registers, buses
2. The fetch–execute cycle, and tracing a program on a simple machine (MARIE)
3. Von Neumann vs Harvard architectures, and our Hack computer
4. Part II – Instruction set architectures (ISAs): what an ISA defines
5. Stack, accumulator and register architectures; number of addresses
6. Addressing modes; RISC vs CISC; where Hack fits

S1.W6.L1: PART I How a computer runs a program

Recap: Structure

Figure 1.1 provides a hierarchical view of the internal structure of a traditional single-processor computer

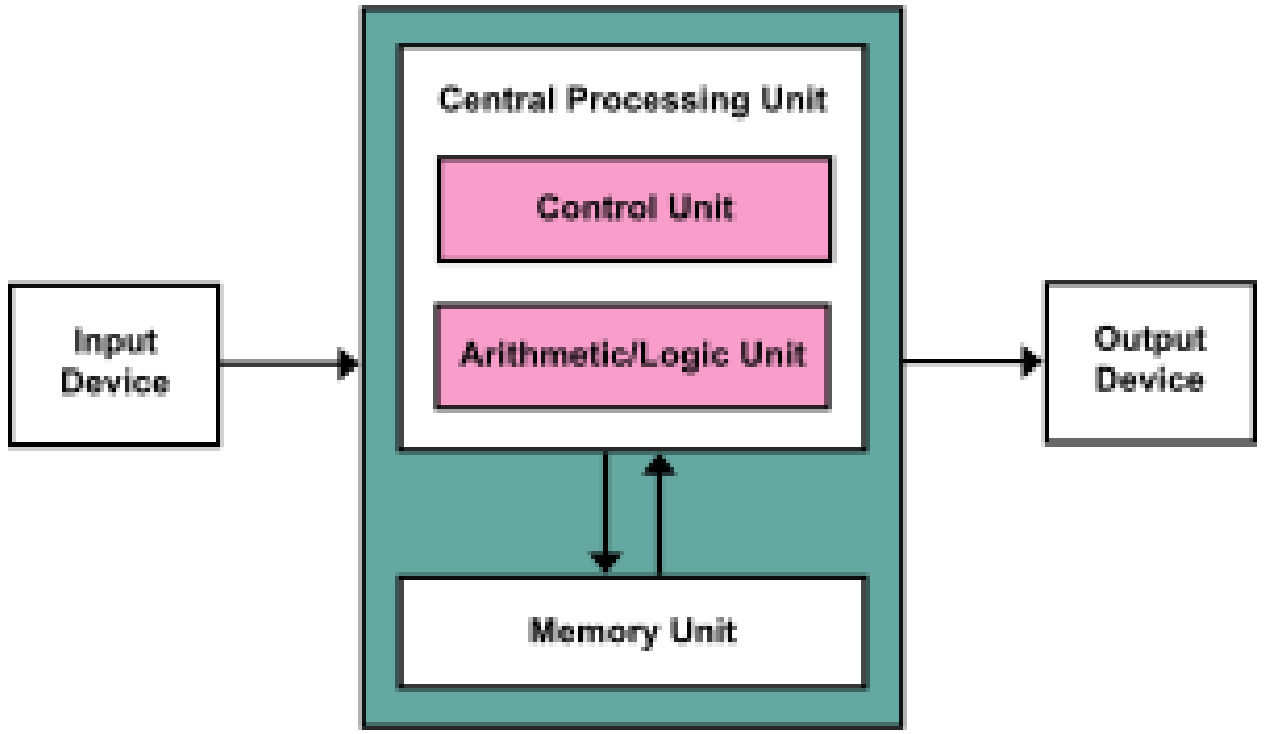
传统单处理器计算机的层次结构图



Source: Stallings. *Computer Organisation & Architecture.* p.27

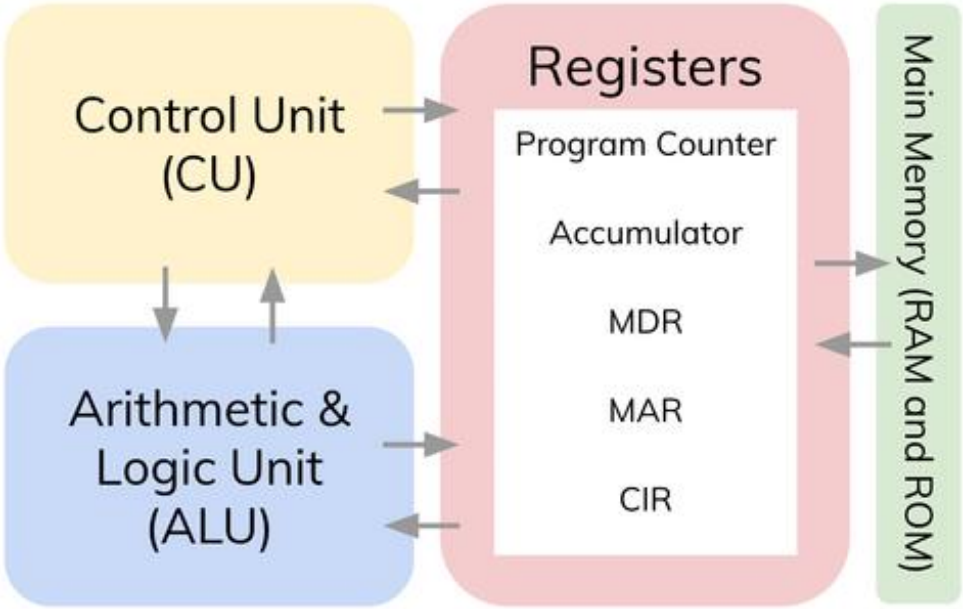
Recap: von Neumann Architecture（图要背）

- The von Neumann machine is based on four linked components（基于4种相互连接的组件）：
  - Main memory which can store programs alongside data.
  - An Arithmetic and Logic Unit (ALU) capable of operating on binary data.
  - A control unit which interprets instructions in memory and executes them.
  - Input and output equipment operated by the control unit.



Recap: Registers （图要背）

- Memory address register (MAR)
  - Specifies the address in memory of the word to be written from or read into the MDR
- Memory data register (MDR)
  - Contains a word to be stored in memory or sent to the I/O unit
  - Or is used to receive a word from memory or from the I/O unit
- Current Instruction register (CIR)
  - Contains the opcode instruction being executed
- Program counter (PC)
  - Contains the address of the next instruction to be fetched from memory
- Accumulator (AC)
  - Employed to temporarily hold operands and results of ALU operations



General Purpose Registers（通用目的寄存器）

- Data:
  - Like accumulators
- Addressing:
  - Hold addresses of memory locations
- How many?
  - Typically between 8 to 32 (128 in Intel Itanium)
- How big?

Large enough to hold full address/word
But we can use two data registers for one word/address

Program Status Word Register （PSW）

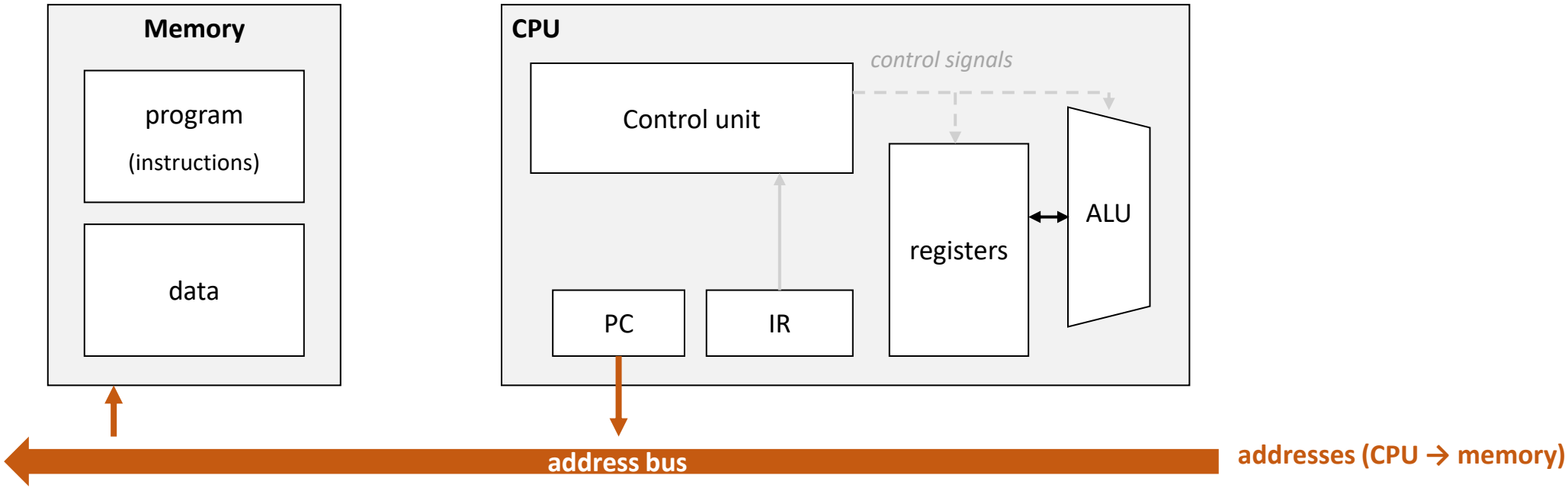
- PSW stores a set of bits



An 8-bit PSW

- Typically including condition codes like
  - Zero flag: set when the result of an arithmetic/logical operation is zero
    - E.g., to test whether a = b, compute a − b and check whether the zero flag is set
  - Carry
    - Enables numbers larger than a single word to be added/subtracted by carrying a binary digit
  - Overflow
  - Interrupt enable/disable
  - Interrupt flags

System bus: address bus（单向：CPU->内存）

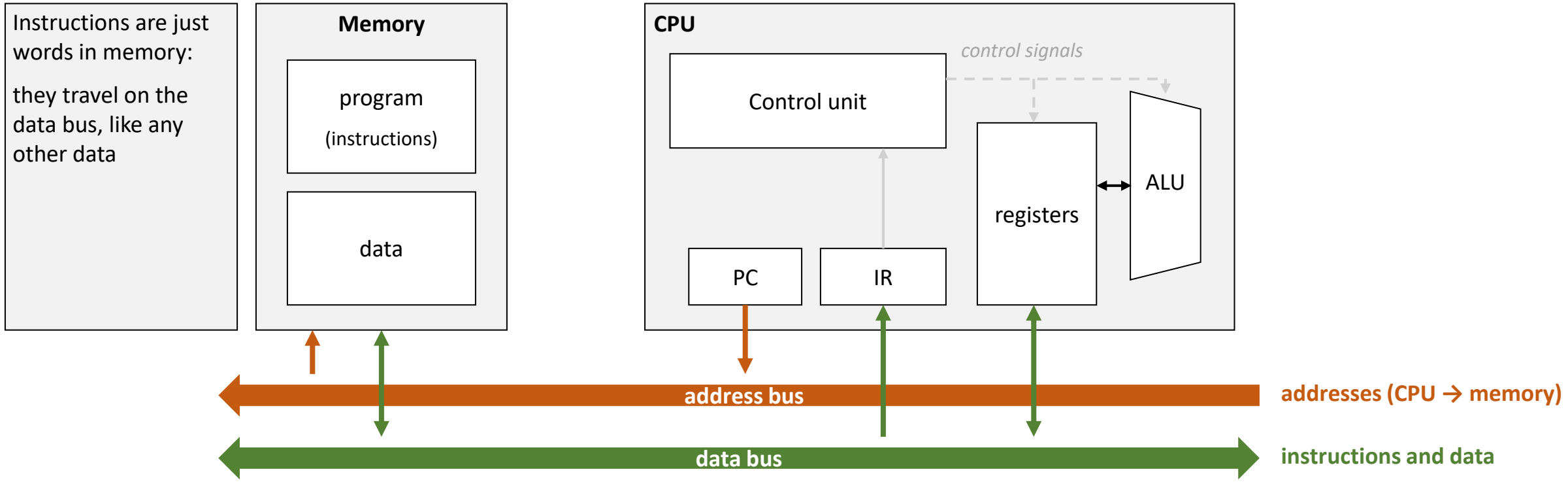


地址线用于指定数据总线上数据的来源或去向，地址线通常也用于I/O端口的寻址。

- CPU将所需要的地址放在地址线上。

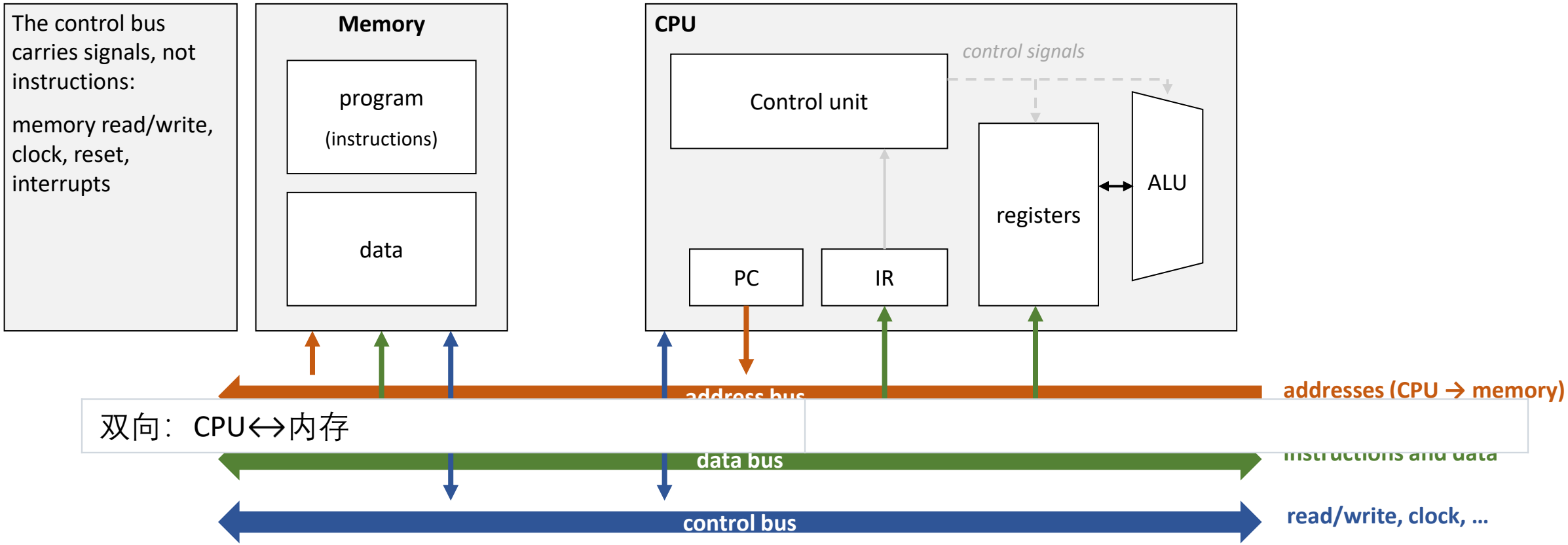
- 地址线的高位用于选择总线上指定的模块，低位用于选择模块内具体的存储器单元或1/0端口。
  例如,在一个8位地址总线上,小于等于01111111的地址可以用来访问有128个字的存储器模块 (模块0),
  而大于等于10000000的地址可用来访问接在I/O模块上的设备(模块1)。

System bus: data bus （双向：CPU↔内存）



- 数据线提供系统模块间传送数据的路径,这些线组合在一起称为数据总线。典型的数据总线包含32、64、128或更多的分离导线, 这些线的数目称为数据总线的宽度。数据总线宽度是决定系统总体性能的关键因素。例如,如果数据总线为32位宽,而每条指令64位长,那么处理器在每个指令周期必须 访问存储器模块两次。

System bus: control bus（双向：CPU↔内存）



- 控制线用来控制对数据线和地址线的存取和使用。由于数据线和地址线被所有模块共享，因此必须用一种方法来控制它们的使用。

- 控制信号在系统模块之间发送命令和时序信号。时序信号指定了数据和地址信号的有效性，命令信号指定了要执行的操作。典型的控制信号:
  memory read/write、I/O read/write、Bus Request/Grant、Interrupt Request/Ack、Clock、Reset….

三种Bus相互协作的例子： CPU读入指定内存地址中的数

阶段1：CPU发起访存请求（控制总线先启动）

CPU的控制单元（CU）首先通过**控制总线**向内存发送一组控制信号：

- **核心命令信号**：发出 Memory Read 命令信号；
- **时序同步信号**：发送Addree Eanble**信号**和全局时钟信号，保证CPU和内存的操作节奏一致；
- 如果内存当前正在处理其他请求，会通过**控制总线**向CPU返回「**BUSY信号**」，CPU会自动等待，直到内存空闲。

阶段2：地址总线指定目标单元

- CPU确认内存空闲后，会把**目标内存地址放到CPU内部的MAR**中，MAR会通过**地址总线**把地址信号发送到内存 的地址引脚。
- 内存的地址译码器(Decoder)会根据地址总线上的信号，将**地址里的数据读到内存的内部数据缓冲器**中，但此时不会直接把数据放到外部数据总线上，等待下一步确认。

阶段3：控制总线确认数据就绪

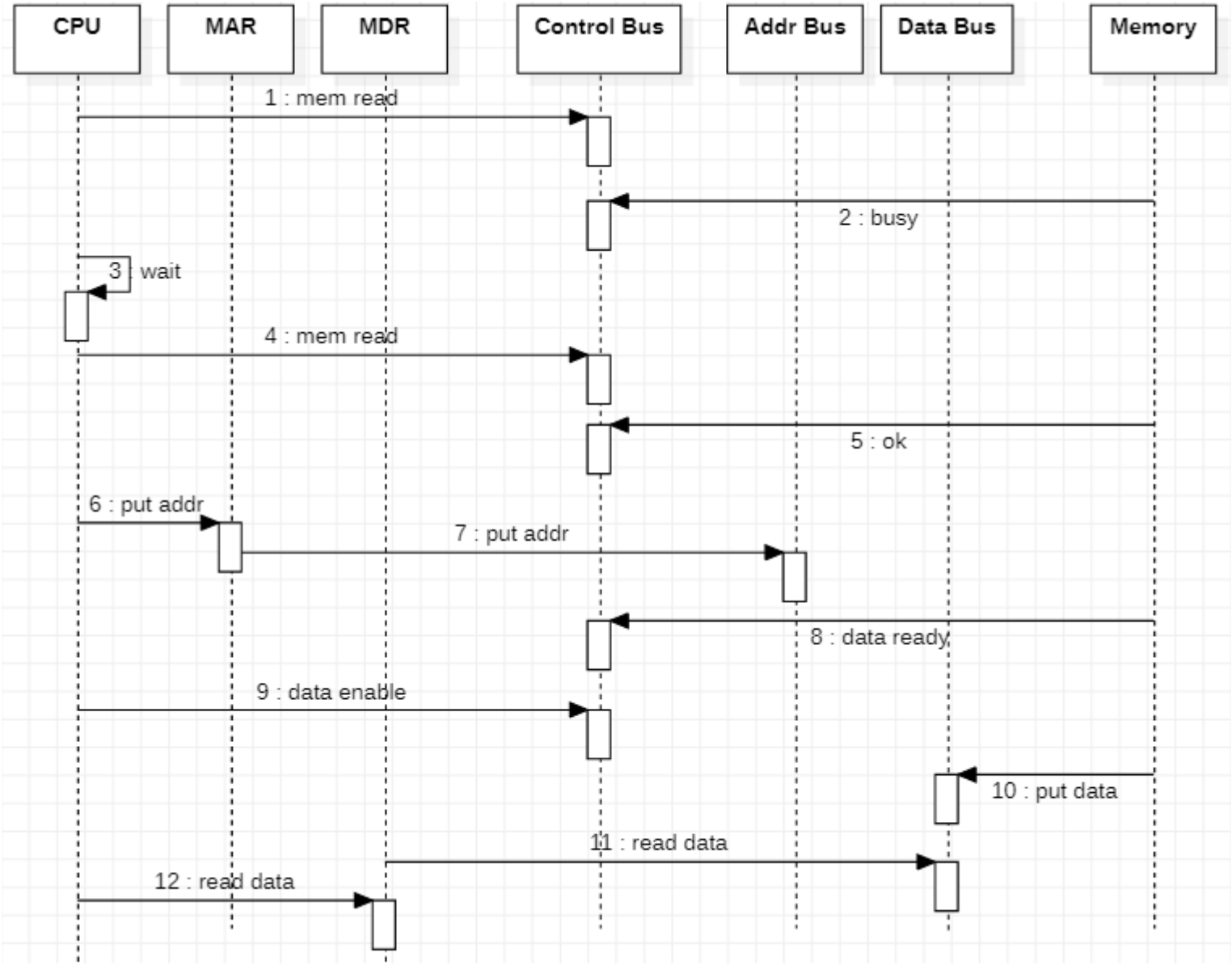
- 内存把数据准备好后，会通过**控制总线**向CPU发送「**Data READY信号**」，相当于告诉CPU「你要的数据我已经拿到了，可以收了」。

阶段4：数据总线传输实际数据

- CPU检测到READY信号有效后，会通过**控制总线**发送「**数据接收使能信号**」，此时内存才会把缓冲好的数据放到**数据总线**上。 CPU内部的MDR会在时钟信号的边沿，锁存数据总线上的数值，完成「内存→CPU」的数据传输，至此三总线的外围交互已经全部完成。

阶段5：CPU内部传输到累加器（三总线不再参与）

三种Bus相互协作的例子：CPU读入指定内存地址中的数



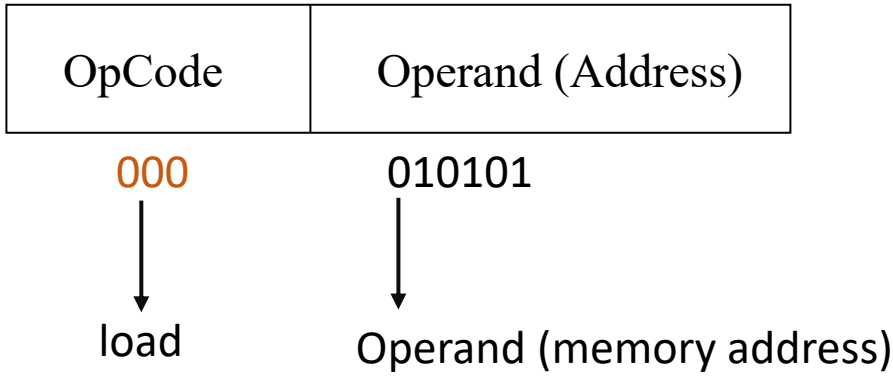
Programming

- Configure the hardware to carry out a specific task
  - Control signals are supplied along with the data to execute the operations（控制信号与数据一起提供以执行操作）
- Programs are made up of instructions（程序由指令组成）
  - CPU executes one instruction every step (or clock cycle)
    - Modern CPUs do more
  - For each step, an arithmetic or logical operation is performed on some data.

Program instructions

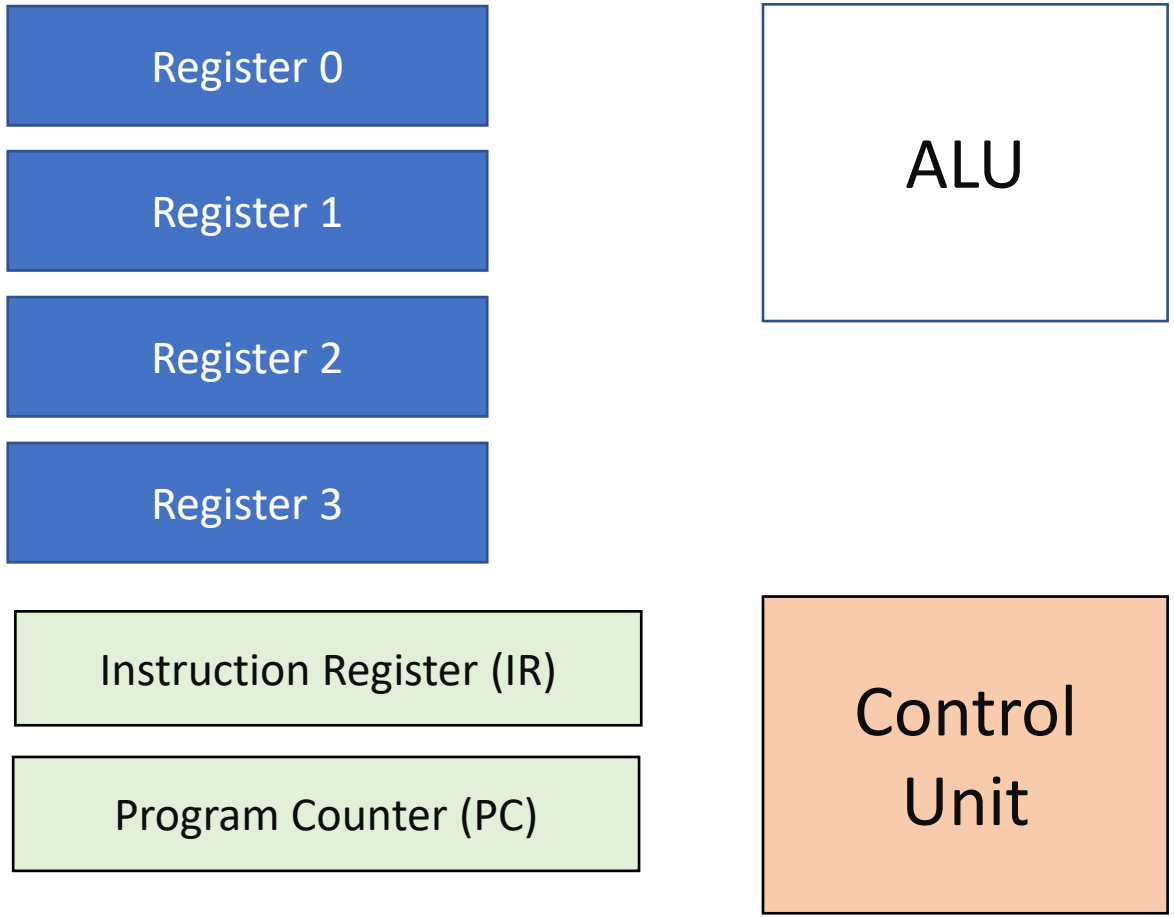
- A program instruction consists of
  - opcode (operation code)
  - zero or more operands
    - immediately available data (embedded within the instruction)
    - the address where the data can be found in main memory or registers
- The most significant bits provide the opcode (最重要的位提供操作码)
- The remaining bits give operands

Opcode	Binary rep,
Load	000
Store	001
Add	010
Sub	011
Jump	100



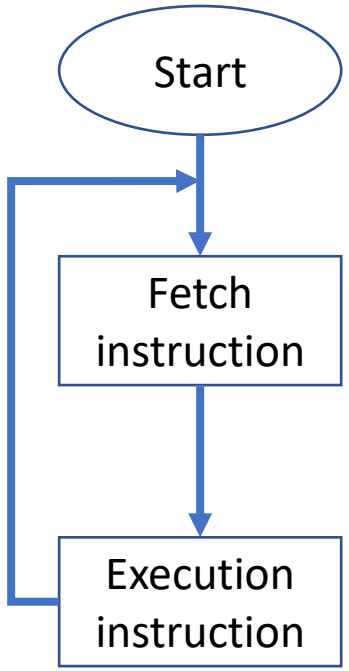
The control unit

- It all comes down to the control unit
- This is just a state machine(状态机)



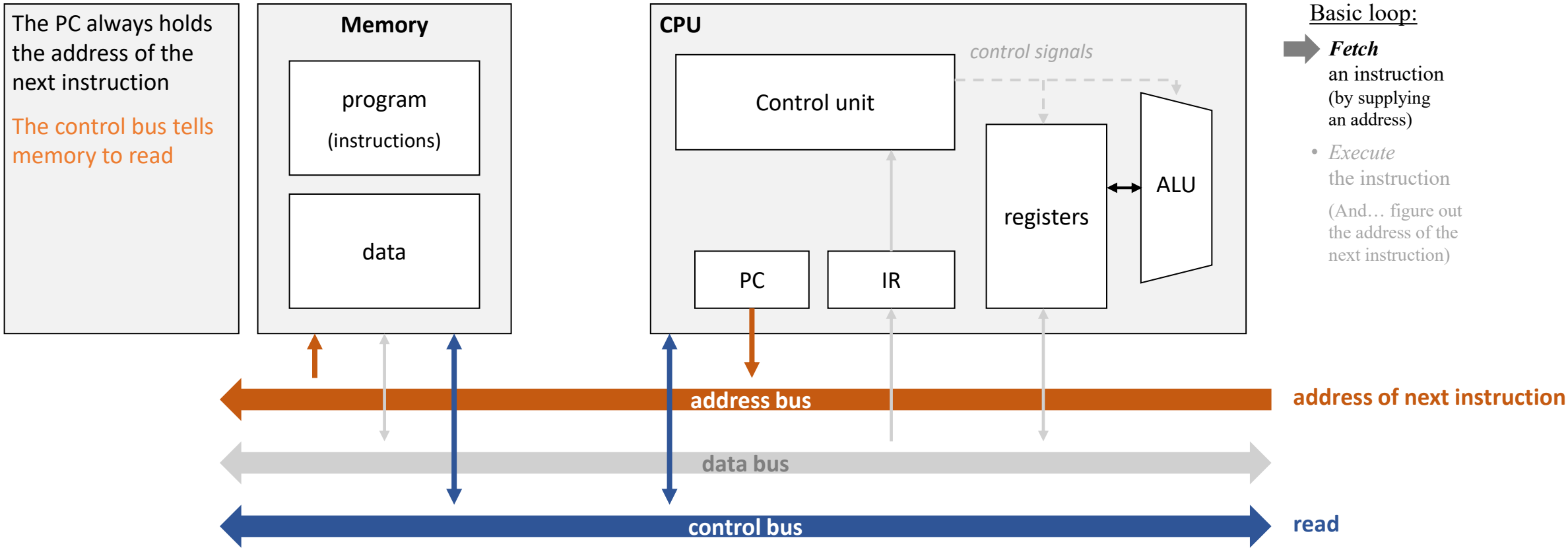
The control unit (cont.)

- Fetch:
  ask the main memory (RAM) for the instruction whose address is stored in program counter (PC)
- Increase PC:
  increase the address stored in PC by 1
- Execute:
  There are only a small number of possible instructions. Depending on which it is, do what is necessary to execute it.
- Repeat
  - Go back to step 1



Fetch (1): the PC addresses memory

Fetch: [pc] -> IR, pc + 1



Basic loop:

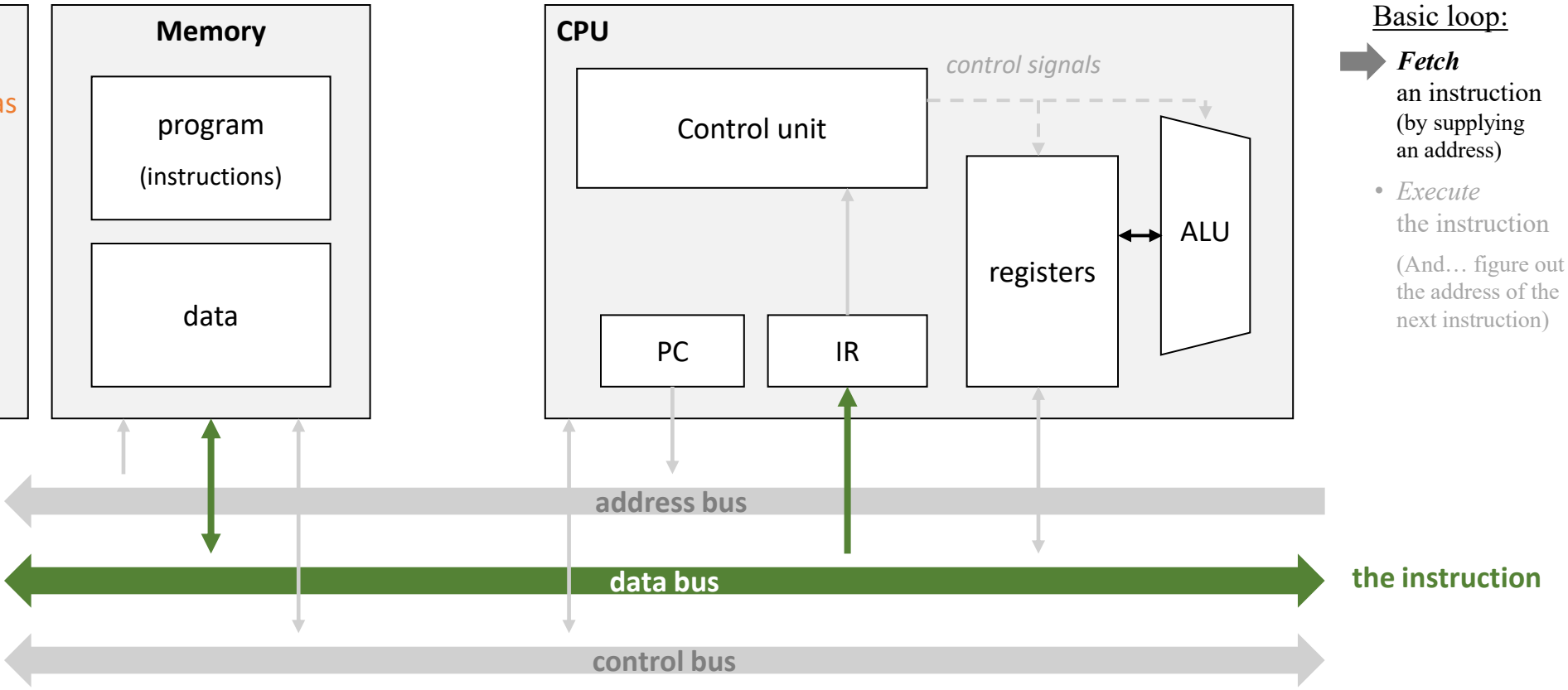
- ***Fetch*** an instruction (by supplying an address)
- *Execute* the instruction

  (And… figure out the address of the next instruction)

Fetch (2): the instruction arrives in the IR

The instruction travels on the data bus, the same path as data

This shared path is the von Neumann bottleneck



Basic loop:

- ***Fetch*** an instruction (by supplying an address)
- *Execute* the instruction

  (And… figure out the address of the next instruction)

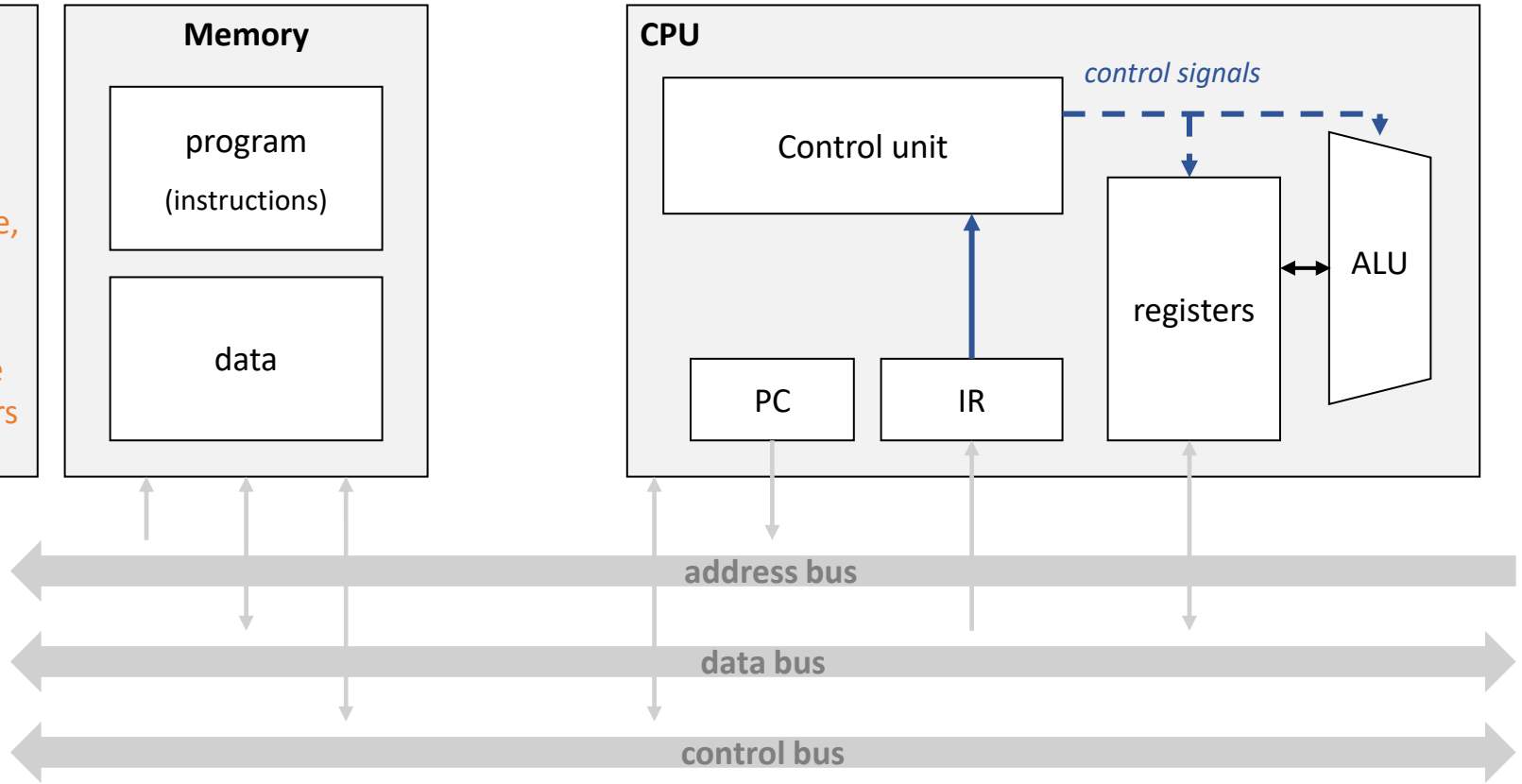
Execute the instruction

1. Decode： 将IR中的指令解码，所谓Decode就是把IR中的二进制数(比如: 1940），分解出opercode： 1和operrand ： 940，最后翻译成指令： load [940]
2. Execute： CU执行load [940]指令，将地址940中的值放入ACC

The control unit decodes the IR

The instruction bits specify which operation to execute, and which memory address it acts on

Control signals drive the ALU and registers



Basic loop:

- *Fetch* an instruction (by supplying an address)
- ***Execute*** the instruction

(And… figure out the address of the next instruction)

A Simple Instruction Set

Instruction Number	Instruction	Meaning
1	Load X	Load content of address X in AC
2	Store X	Store the contents of AC at address X
3	Add X	Add the contents of address X to AC and store result in AC
4	Subt X	Subtract the contents of address X from AC and store result in AC
5	Input	Input a value form keyboard into AC
6	Output	Output the value of AC to the display
7	Halt	Terminate the program
8	Skipcond	Skip the next instruction on condition
9	Jump X	Load the value of X into the PC



将16-bits拆成4-bits和12bits
4-bits放OpCode， 12-bits放Operand

MARIE's instruction set, from L. Null and J. Lobur 2014, The Essentials of Computer Organization and Architecture

Example: tracing a MARIEprogram

Memory address	Memory value
300	1940
301	3941
302	2941
…	…
940	0003
941	0002

Opcodes:

1 : Load AC from memory

2 : Store AC to memory

3 : Add to AC from memory

PC initially has a value of 300

Note: values are in hexadecimal – one hex digit of opcode followed by three hex digits of address (4 + 12 bits), exactly MARIE's format(指令集见上页).

Example (cont.)

- **Fetch:** place the contents of address 300 in the IR
  - IR = 1940
- **Increment** the contents of PC
  - PC = 0301
- **Execute**: interpret and carry out the instruction held in IR,

  i.e. load the contents of memory(OpCode: 1) address 940 into AC
  - AC = 0003
- **Fetch**: place the contents of address 301 in the IR
  - IR = 3941
- **Increment** the contents of PC
  - PC = 0302
- **Execute**: interpret and carry out the instruction held in IR, i.e. add the contents of memory address 941 into AC(OpCode: 3)
  - AC = 0005

Memory address	Memory value
300	1940
301	3941
302	2941
…	…
940	0003
941	0002

Instruction Number	Instruction
1	Load X
3	Add X

Memory address	Memory value
300	1940
301	3941
302	2941
…	…
940	0003
941	0002

- **Fetch**: place the contents of address 302 in the IR

  IR = 2941

- **Increment** the contents of PC

  PC = 0303

- **Execute**: interpret and carry out the instruction held in IR, i.e., store the contents of AC in memory address(OpCode: 2)

  Mem[941] = 0005

Instruction Number	Instruction
2	Store X

Your Turn: What would the following program do?

Memory address (Hex)	Memory value (Bin)	Memory value (Hex)
100	0001000100000100	1104
101	0011000100000101	3105
102	0010000100000110	2106
103	0111000000000000	7000
104	0000000000100011	0023
105	1111111111101001	FFE9
106	0000000000000000	0000

Program Counter starts at 100
Values in memory are stored in two's complement

Your Turn: What would the following program do?

Step	PC	IR	AC	
(initial values)	100	-	-	
Fetch	100	1104	-	Fetch instruction from address 100
Execute	101	1104	0023	Load AC with contents of address 104 (35)
Fetch	101	3105	0023	Fetch instruction from address 101
Execute	102	3105	000C	Add contents of 105 (-23) to AC
Fetch	102	2106	000C	Fetch instruction from address 102
Execute	103	2106	000C	Store contents of AC in address 106 (12)
Fetch	103	7000	000C	Fetch instruction from address 103
Execute	104	7000	000C	Halt

Memory address (Hex)	Memory value (Bin)	Memory value (Hex)
100	0001000100000100	1104
101	0011000100000101	3105
102	0010000100000110	2106
103	0111000000000000	7000
104	0000000000100011	0023
105	1111111111101001	FFE9
106	0000000000000000	0000

	Instruction
1	Load X
2	Store X
3	Add X
5	Input
6	Output
7	Halt

Your Turn: What would the following program do?

Memory address (Hex)	Memory value (Bin)	Memory value (Hex)
100	0001000100000100	1104
101	0011000100000101	3105
102	0010000100000110	2106
103	0111000000000000	7000
104	0000000000100011	0023
105	1111111111101001	FFE9
106	0000000000001100	000C



Adds 0x0023 and 0xFFE9 and stores result (0x000C) in address 106

In decimal: Adds 35 and -23 and stores result (12) in address 106

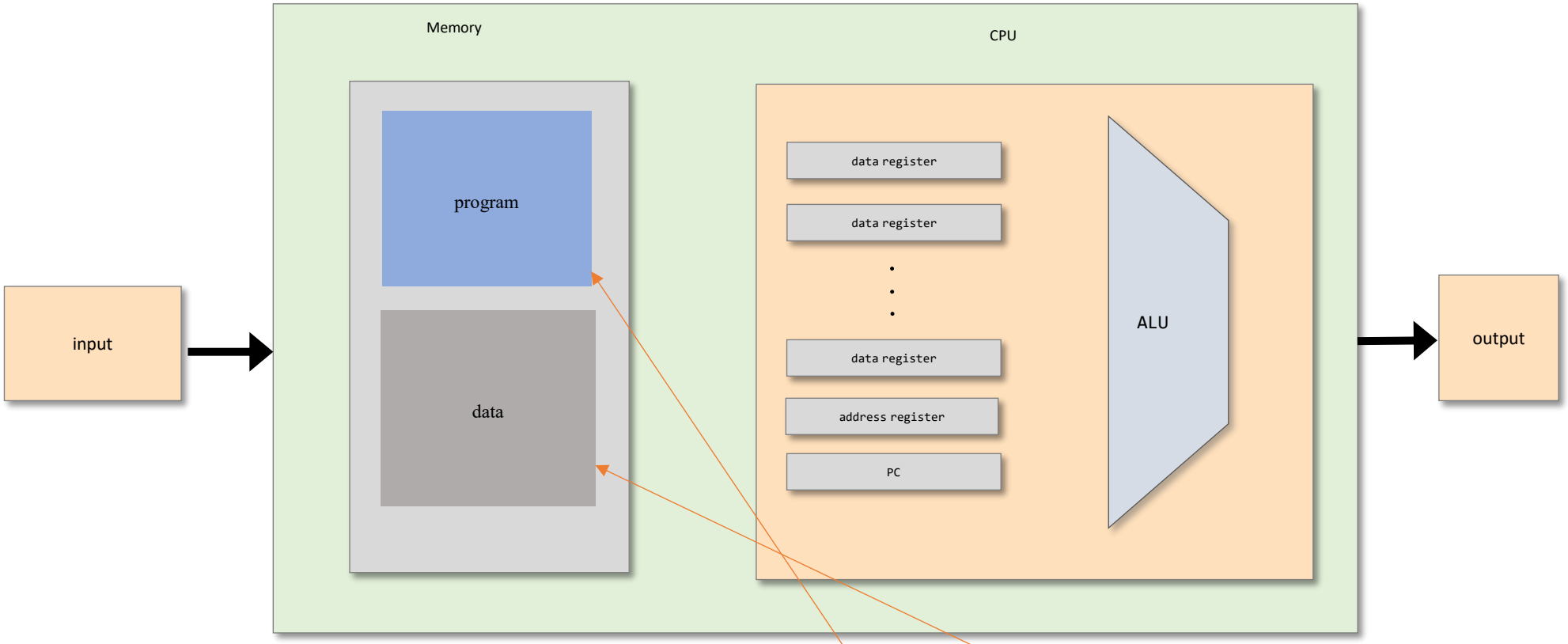
Program Counter starts at 100
Values in memory are stored in two's complement



https://indrusiak.github.io/files/COMP1860/instruction-set-simulator.html

HACK Computer Architecture

- 计算机要素： P62

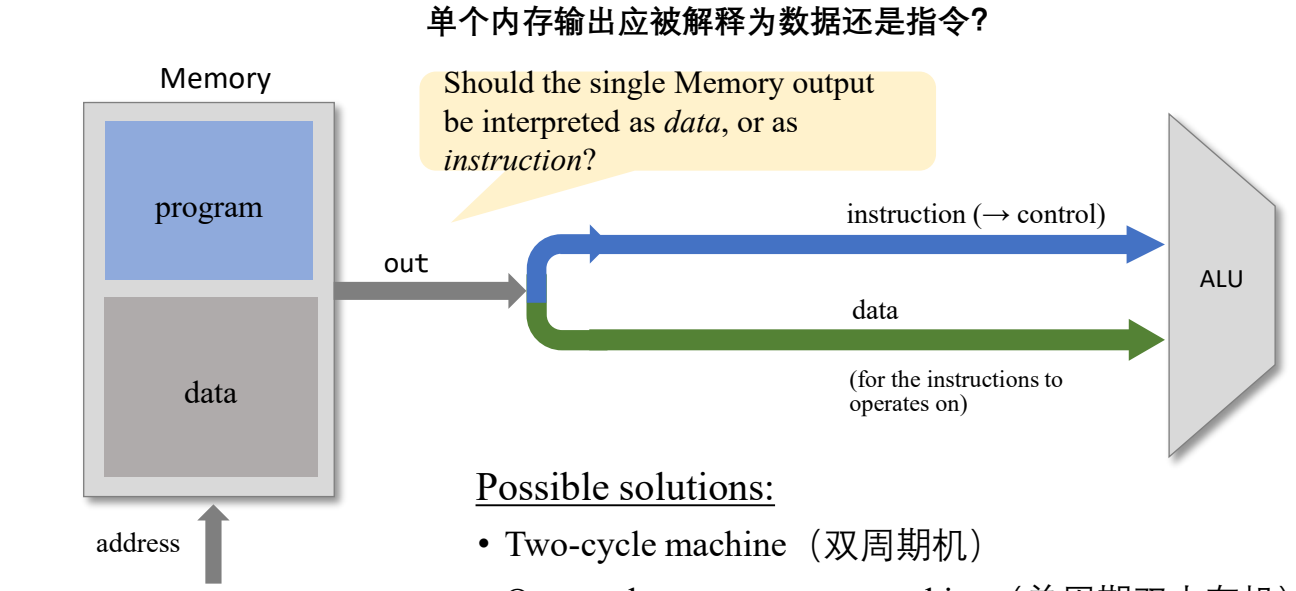


Hack平台是一个16位的冯·诺依曼机,专门用于执行使用Hack机器语言编写的程 序。
Hack平台由一个CPU、两个单独的存储器模块(分别用作指令存 储器和数据存储器),
以及两个内存映射的I/O设备(屏幕和键盘)组成。
和冯诺曼体系不同的是，指令和数据是分开存放的。

HACK Computer Registers

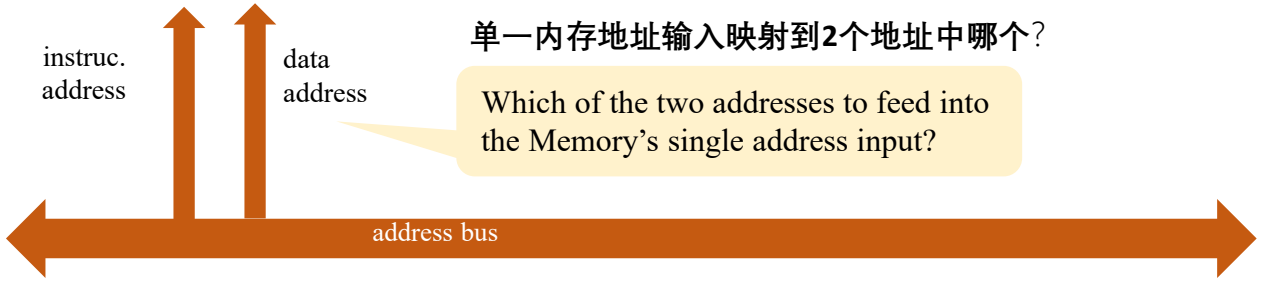
- Address register (A)
  - Specifies the address in memory of the word to be written from or read into (指定要写入或读入的字在内存中的地址)
  - OR can be used as a Data Register （或者当数据寄存器使用）
- Data Register (D)
  - Contains a word to be stored in memory or sent to the ALU
- Program counter (PC)
  - Contains the address of the next instruction to be fetched from (instruction) memory

Fetch – Execute Issues

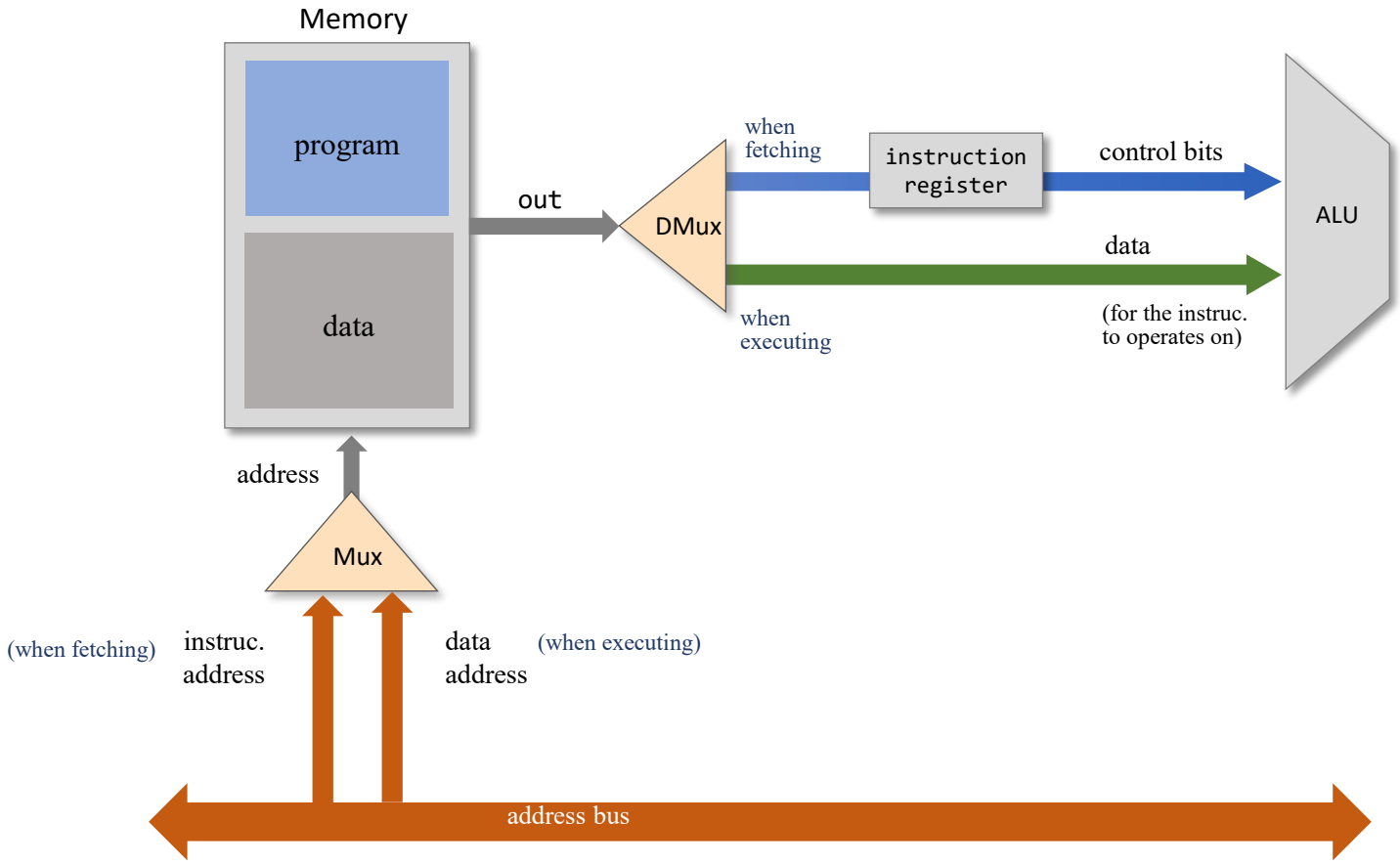


Possible solutions:

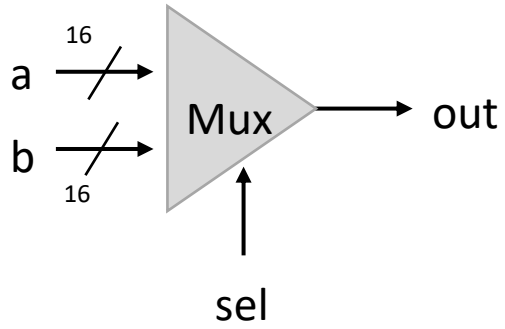
- Two-cycle machine（双周期机）
- One-cycle, two-memory machine（单周期双内存机）



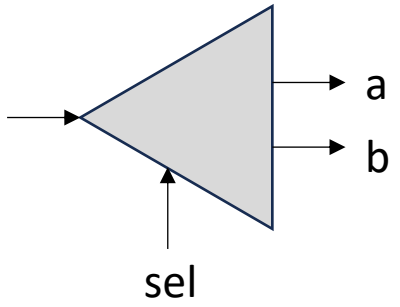
Two-Cycle Solution（双周期解决方案）



Mux: 信号选择器



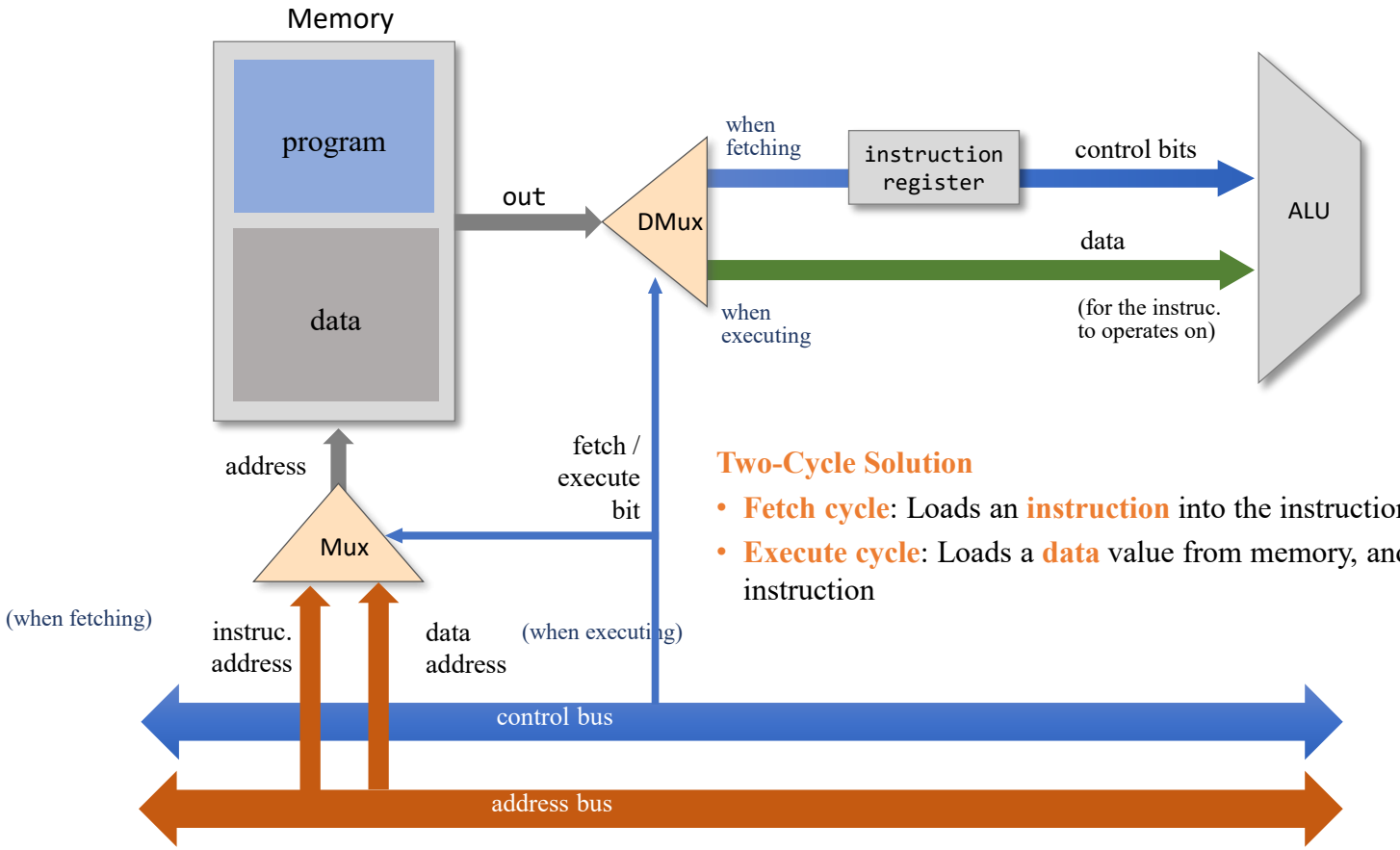
Dmux：反向信号选择器



Two-Cycle Solution：

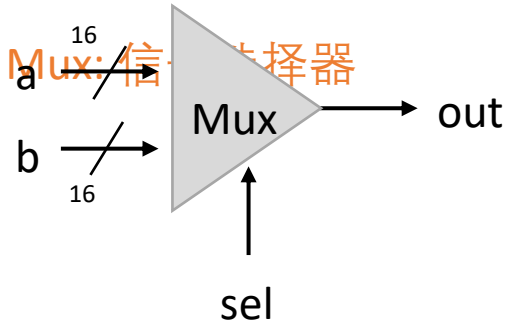
- Fetching cycle用于instruction address
- Executing cycle用于data address

Two-Cycle Solution

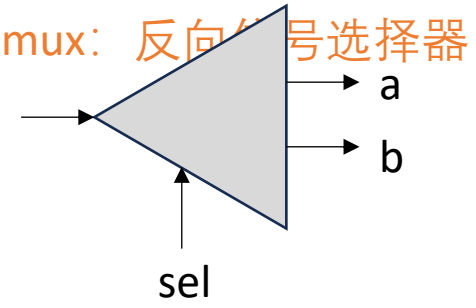


Two-Cycle Solution

- **Fetch cycle**: Loads an **instruction** into the instruction register
- **Execute cycle**: Loads a **data** value from memory, and executes the instruction



Mux：信[illegible]选择器



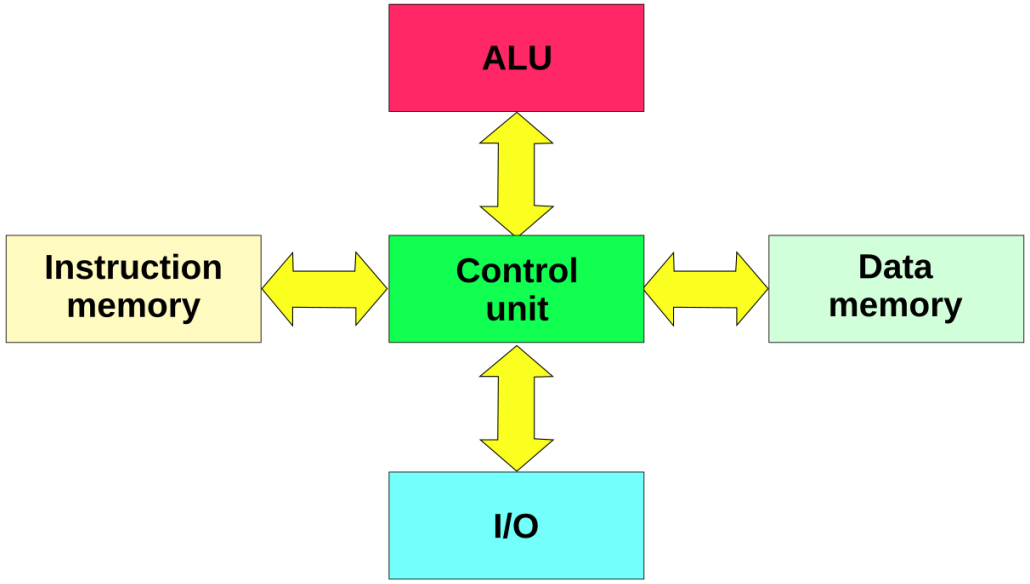
Dmux： 反向[illegible]号选择器

Von Neumann Bottleneck（瓶颈）

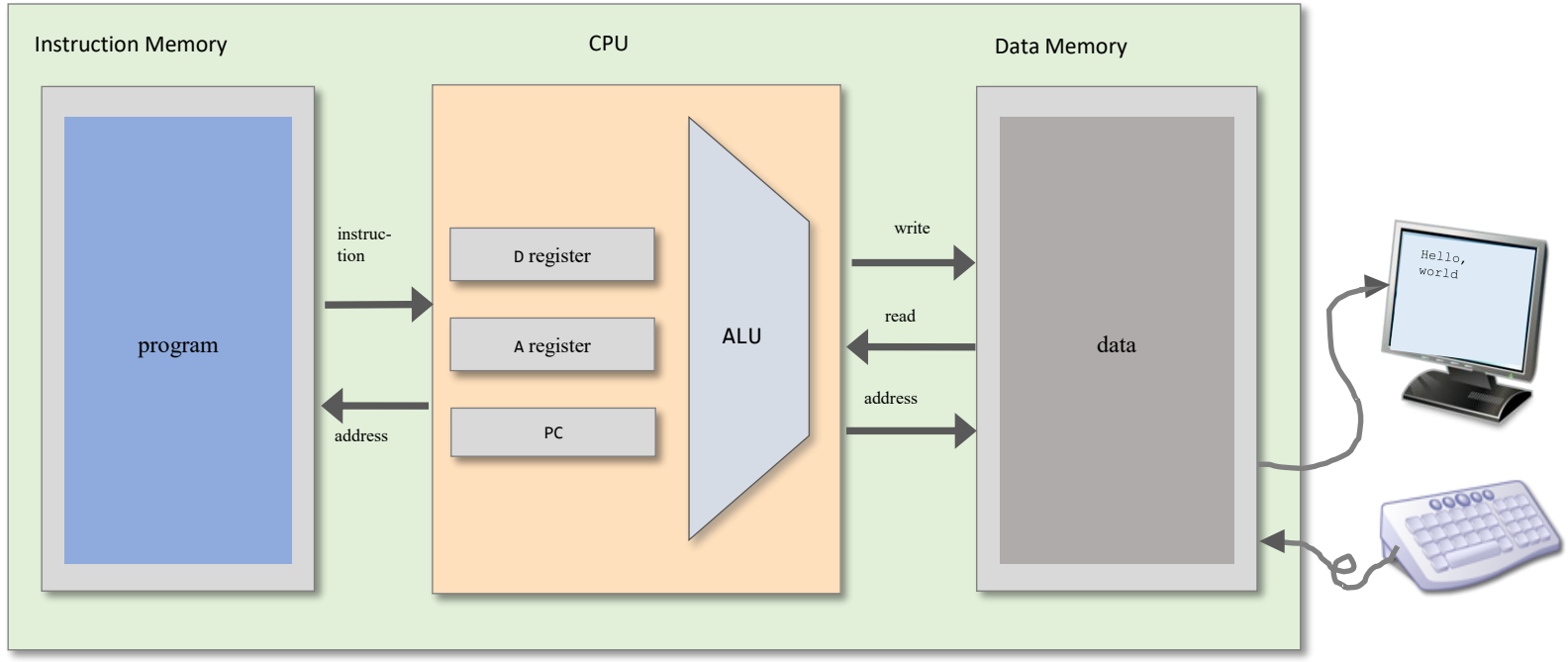
- Von Neumann architectures shares memory bus for data and program instructions
- Single bus limits throughput (data transfer rate) between CPU and memory
- CPU must wait for data to be moved to or from memory

Harvard Architecture

- Named for Harvard Mark 1 (1944)
- Difference is two buses from memory to CPU
  - Instructions
  - Data
  - Mark 1 could read in instructions and data from punch cards simultaneously（从穿孔卡片）
- Advantages
  - CPU receives instruction and data concurrently（同时）
- Disadvantages
  - Programs cannot be stored in same memory as data



HACK: Single cycle, two-memory machine



- Single cycle： 单周期
  所有Hack指令，都严格在1个时钟周期内完成从取指、译码、ALU运算、结果写回的全部操作，不需要拆分到多个周期

- Two memory：双内存/指令-数据分离存储（并非冯诺曼体系的单一内存架构: 数据和指令在同一内存中）
  Instruction Memory, IM：只读ROM，存储整个程序的二进制指令，运行时仅支持读取、无法修改，程序启动时一次性烧录。
  Data Memory, DM： RAM，存储程序的变量、堆栈数据，还做了内存映射I/O：Hack平台的屏幕、键盘等外设的寄存器都挂在DM的地址空间里，读写对应地址等同于操作外设。

- HACK机器采用的是Harvard Architecture（哈佛架构）

HACK: Single cycle, two-memory machine

特性	Single cycle, two-memory（本架构）	Two-cycle, one-memory（你之前问的双周期机器）	单周期+one-memory（不存在的冲突架构）
内存数量	2个（IM只读+DM可读可写）	1个（统一存储指令和数据）	1个
执行效率	1周期/指令，吞吐量最高	2周期/指令，吞吐量减半	理论上1周期，但存在硬性访存冲突无法实现
硬件开销	需要额外一个内存模块，面积更大	仅需单个内存，面积更小	无
时序复杂度	极低，无周期状态控制，所有操作同步	稍高，需要区分取指/执行周期的状态逻辑	无法实现

之所以单周期Hack必须用双内存，是因为如果指令和数据共用同一个内存（one-memory），同一个周期CPU既要读内存取指、又要读/写内存做数据操作，单一内存同一时间只能响应一个访问请求，必然产生冲突，因此单周期架构只能搭配双内存使用；而双周期机器则是反过来，用1个内存，把取指和数据访问拆分到两个周期，避开同时访存的问题，用时间换面积。

Your turn: A simple program

c ← a +b

Instructions in program for HACK architecture:

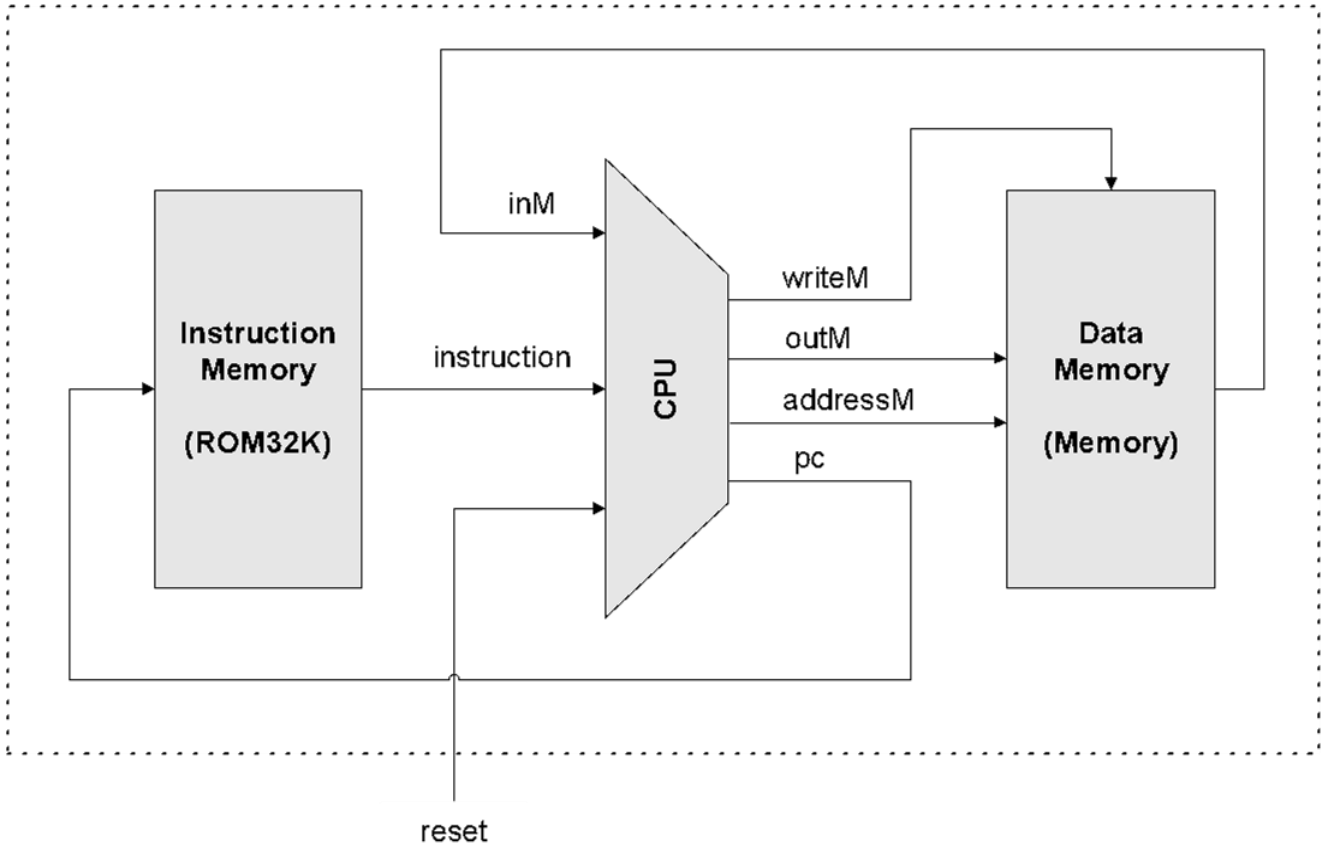
Your turn: A simple program

c ← a +b

Instructions in program for HACK architecture:

- Load value in a into D register
- Load value in b into A register
- D ← D + A
- Store D in address c

Proposed Computer Architecture（拟议的计算机体系结构）



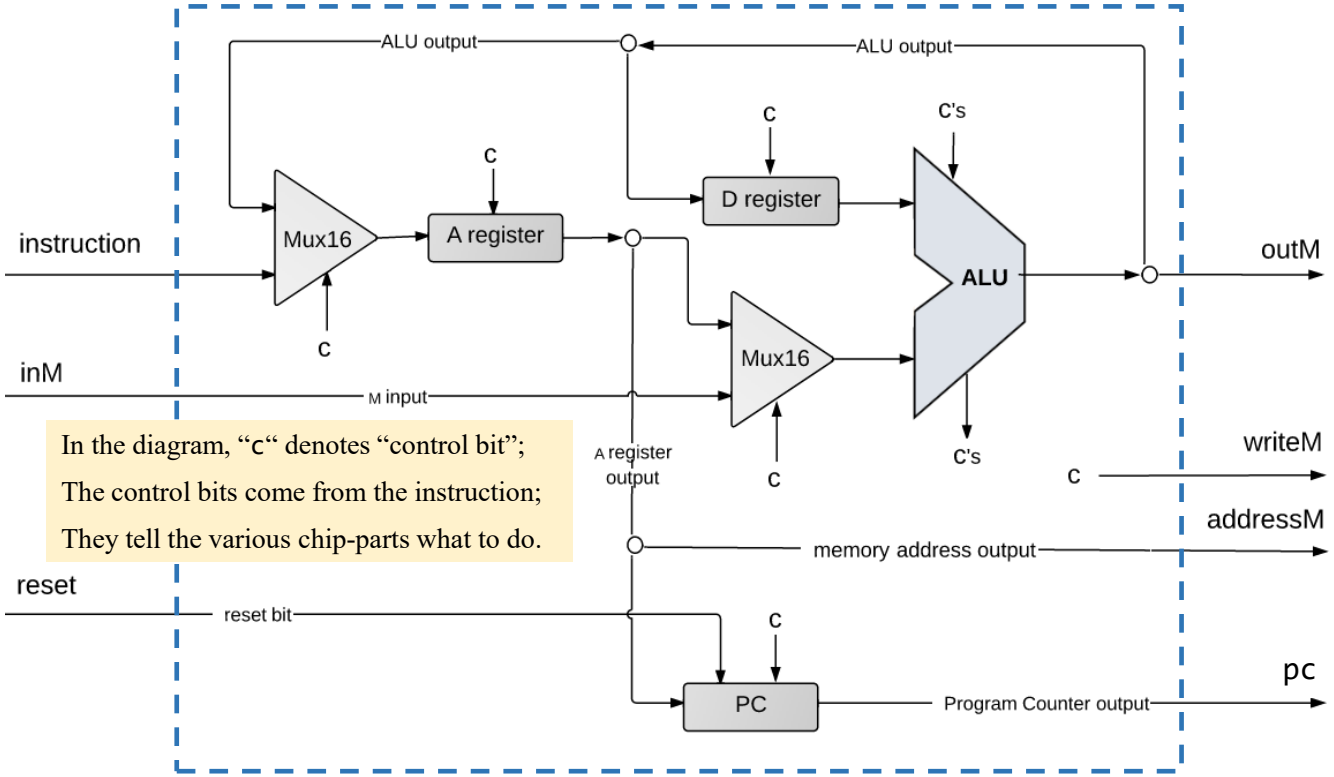
1. 当用户输入reset置1时,CPU 的输出pc发出0,使指令存储器(ROM32K)输出程序中的第一条指令。
2. CPU将执行该指令,这个执行过程可能包括读取或写入数据存储寄存器。
3. 在该指令的执行过程中,CPU会 确定下一条要提取的指令,并通过其输出pc发出该地址。
4. CPU的输出pc送到指令存储 器的输入address,后者会输出下一条应该执行的指令
5. 这个输出又会送到CPU的输入instruction,从而完成本次取指-执行周期。

Hack平台由一个CPU、两个单独的存储器模块(分别用作指令存 储器和数据存器)组成。
Hack计算机执行存储在指令存储器中的程序。在Hack平台的物理实现中:

- 指令存储 器可以用只读存储器(ROM)芯片实现,ROM芯片可以预先加载好所需的程序。

CPU implementation

计算机要素：P67



In the diagram, "c" denotes "control bit";
The control bits come from the instruction;
They tell the various chip-parts what to do.

- Load value in a into D register
- Load value in b into A register
- D ← D + A
- Store D in address c

S1.W6.L1: PART II Instruction set architectures

计算机组成与体系结构 P238

Instruction set architecture

"Instruction Set Architecture (ISA) is the structure of a computer that a machine language programmer (or a compiler) must understand to write a correct (timing independent) program for that machine."

IBM, Introducing the IBM 360 (1964)

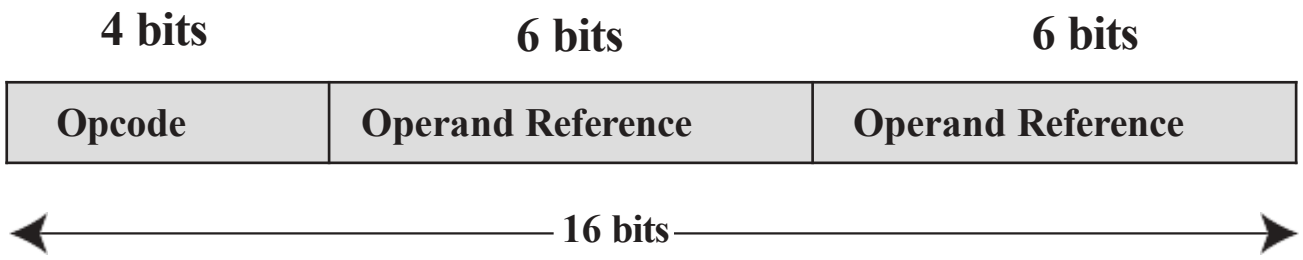
The ISA defines

- Operations that the CPU can execute
- Data transfer mechanism + how to access data
- Control mechanism (branch, jump etc.)
- "Contract" between programmer/compiler and hardware

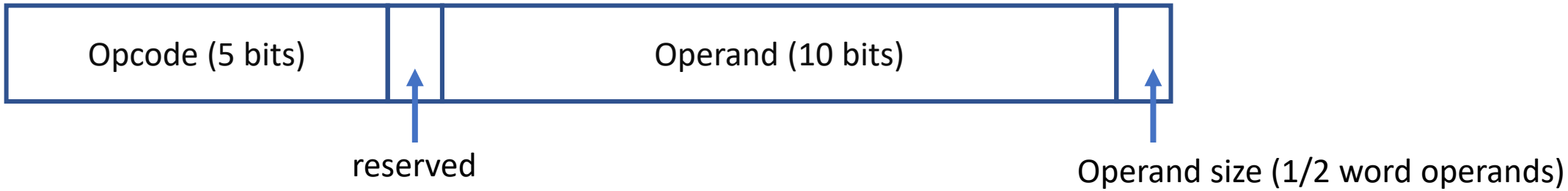
Instruction Number	Instruction	Meaning
1	Load X	Load content of address X in AC
2	Store X	Store the contents of AC at address X
3	Add X	Add the contents of address X to AC and store result in AC

Elements of an instruction

- Operation code (Opcode)
  - Do this
- Source operand reference
  - From this
- Result operand reference
  - Put the answer here
- Next instruction reference（jmp addr这样的指令）
  - When you have done that, do this…



An early ISA: EDSAC (1949)



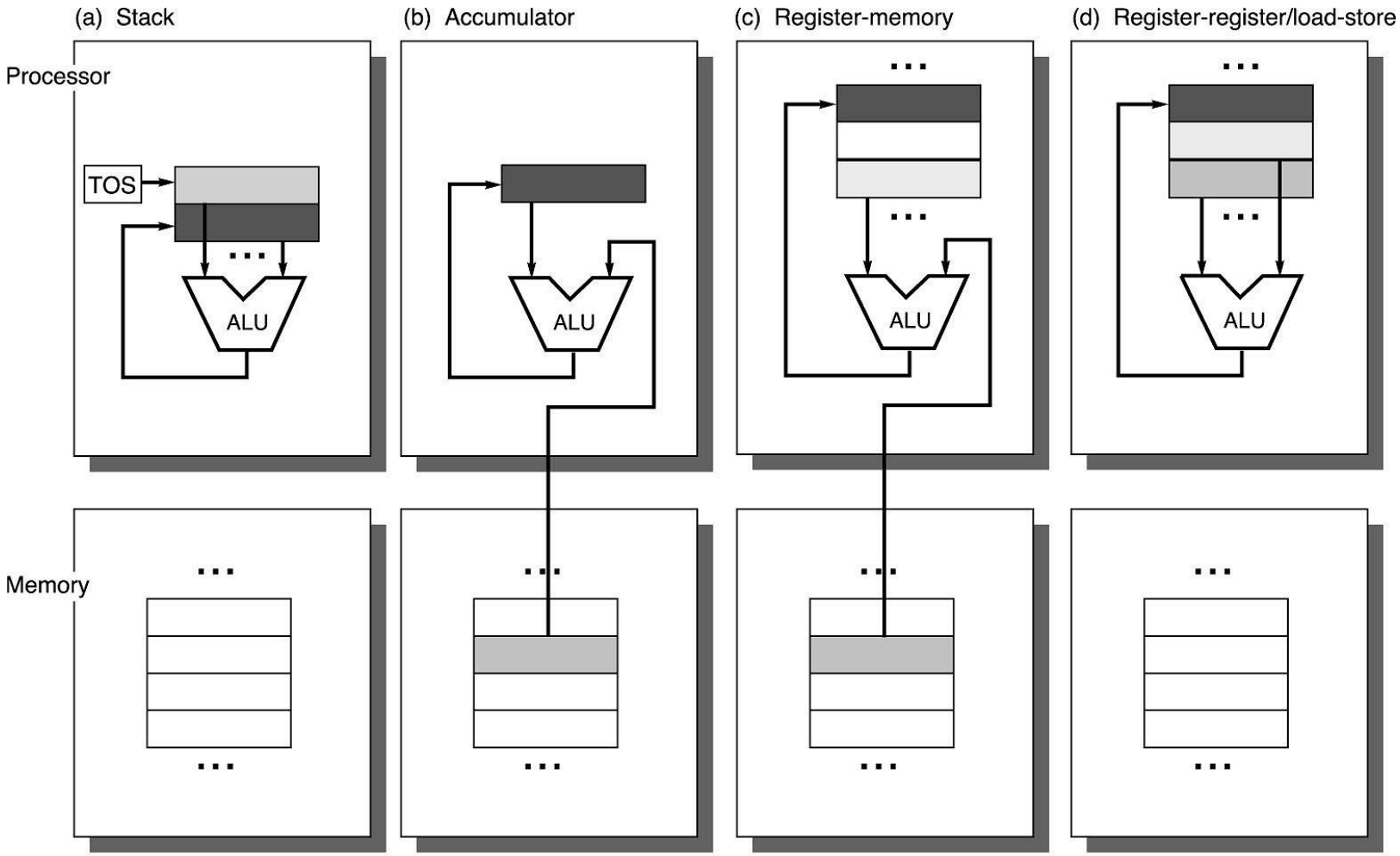
Single accumulator architecture: e.g. AC <- AC + Mem[n]

单累加器架构：用1个累加器和1个内存数进行运算，结果放回累加器

Instruction examples:

- A n: add mem[n] into AC
- T n: transfer the content of AC to mem[n]
- E n: if AC>0, branch to mem[n], else proceed serially
- I n: read the next character from paper tape, and store it in mem[n]
- Z: halt the machine and ring the warning bell.

Classifying ISAs: how ALU operates on data（逻辑运算单元如何操作 数据）



a: Stack:

stack[top-1] = stack[top-1 + stack[top]

b: Accumulator:

acc = acc + mem[n]

c: Register - memory

r1 = r1 + mem[n]

d: Register – Register

r3 = r1 + r2

Stack architecture(堆栈架构)

- Instruction set:

  add, sub, mult, div, . . .

  push, pop

- **Example C = A+B**

  PUSH A

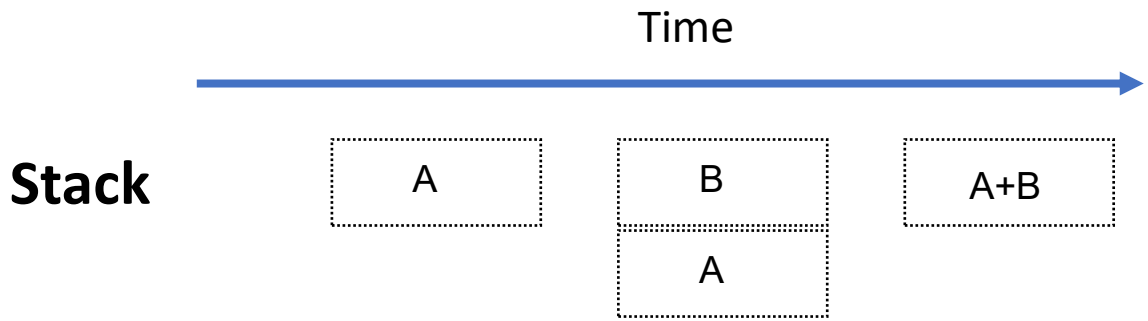
  PUSH B

  ADD

  POP C

x86 floating point operations use stacks

Java virtual machine uses stacks

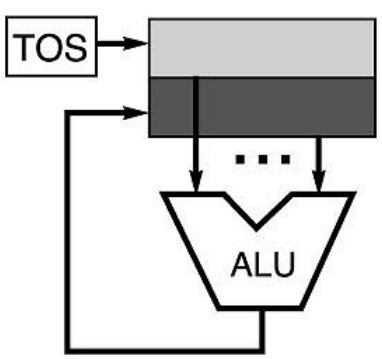


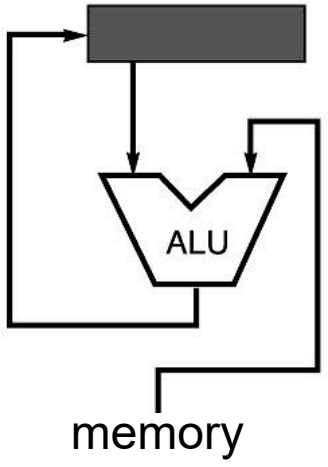
Stack architecture中的ALU运算，没有acc和memory的事，operand（操作数）是用stack来实现的。
比如实现C = A+B：

1. 将 A,B顺序 push堆栈（此时栈顶为B，其下为A。 Stack：后进先出）
2. 在栈顶实现add ， 将结果放回栈顶（此时栈顶为A+B的result）
3. 从栈顶pop结果到c

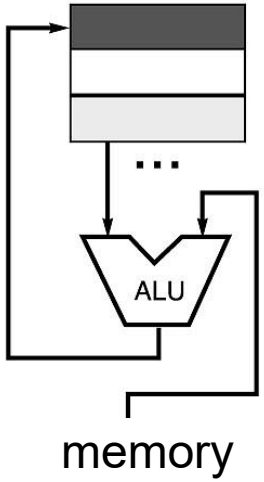
C = A+ B under different ISAs（不同指令集下的C=A+B）

Stack	Accumulator	Register (register-memory)	Register (load-store)
Push A Push B Add Pop C	Load A Add B Store C	Load R1, A Add R1, B Store C, R1	Load R1,A Load R2, B Add R3, R1, R2 Store C, R3

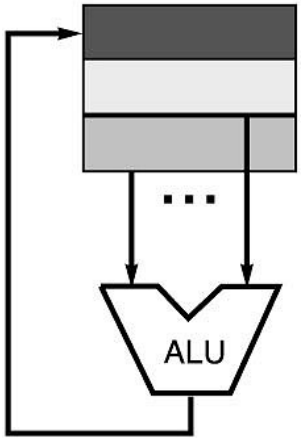




acc = acc + mem[B]



R1 = R1 + mem[B]



R3 = R1 + R2

Stacks

- Pros
  - Compact code (紧凑代码，destination/source is always the top of the stack)
    - Push and Pop operations have 1 explicit operand（1个显式操作数）
  - Low hardware requirements
  - Easy to write a simpler compiler for stack architectures

- Cons

Modern processors use general purpose registers（通用寄存器）

- GPR (General purpose registers) are faster than memory
  - Register values are available immediately
    (gpr在cpu内部，取数不需要addr bus + data bus中转，速度快)

- Registers are convenient for program variables （便于作为程序变量）
  - Compiler assigns some variables just to registers
    （编译器仅将一些变量分配给寄存器）
  - More compact code since small fields (3-8 bits) specify registers (compared to memory addresses)
    代码更紧凑，因为仅需用3~8比特的短字段即可指定寄存器（对比之下，指定内存地址需要更长的字段）

Disadvantages of registers

- Need to save and restore during function calls
- Fixed size (e.g., can’t store strings of an arbitrary length)
- Compiler must manage
- Limited number

Number of addresses

Program to execute $Y = \frac{A-B}{C+D\times E}$

Instruction		Comment
SUB	Y, A, B	Y ←A – B
MPY	T, D, E	T ←D ×E
ADD	T, T, C	T ←T + C
DIV	Y, Y, T	Y ←Y ÷ T

(a) Three-address instructions(指令数:4)

Instruction		Comment
MOVE	Y, A	Y ←A
SUB	Y, B	Y ←Y – B
MOVE	T, D	T ←D
MPY	T, E	T ←T ×E
ADD	T, C	T ←T + C
DIV	Y, T	Y ←Y ÷ T

(b) Two-address instructions (指令数:6)

Instruction		Comment
LOAD	D	AC ←D
MPY	E	AC ←AC ×E
ADD	C	AC ←AC + C
STOR	Y	Y ←AC
LOAD	A	AC ←A
SUB	B	AC ←AC – B
DIV	Y	AC ←AC ÷ Y
STOR	Y	Y ←AC

(c) One-address instructions (指令数:8)

Your turn:

- Write out the sequence of one-address instructions required to execute the following using an architecture with only an Accumulator register

$$E = \frac{(A^2 + B)}{(A - D)}$$

Where A,B,D and E are the values stored in memory addresses A,B,D,E respectively.

Assume you only have following possible instructions that act on the accumulator register

LOAD X	SUB X
MPY X	STOR X
DIV X	ADD X

Where X is a memory address

Your turn: solution

- Compute the denominator first and park it in E, which is free until the end:
  - `LOAD A` `AC ← A`
  - `SUB D` `AC ← A − D`
  - `STOR E` `E ← A − D`
  - `LOAD A` `AC ← A`
  - `MPY A` `AC ← A²`
  - `ADD B` `AC ← A² + B`
  - `DIV E` `AC ← (A² + B) / (A − D)`
  - `STOR E` `E ← AC`

$$E = \frac{(A^2+B)}{(A-D)}$$

Your turn:

If the opcodes for these instructions are:

1 LOAD	2 STOR
3 ADD	4 SUB
5 DIV	6 MPY
7 HALT	

The memory addresses of A,B,D and E are 100,101,105 and 106 in Hex respectively and your instructions are 16-bits long and the opcodes are 4 bits

Write out the machine instructions to execute this task (in Hex)

Your turn: solution

- 16-bit instruction = 4-bit opcode + 12-bit address, so each instruction is 4 hex digits
  - 1100 LOAD A
  - 4105 SUB D
  - 2106 STOR E
  - 1100 LOAD A
  - 6100 MPY A
  - 3101 ADD B
  - 5106 DIV E
  - 2106 STOR E
  - 7000 HALT

$$E = \frac{(A^2+B)}{(A-D)}$$

1 LOAD 2 STOR
3 ADD 4 SUB
5 DIV 6 MPY
7 HALT

A,B,D and E are 100,101,105 and 106 in Hex

instructions are 16-bits long and the opcodes are 4 bits

How many addresses?

- **More addresses per instruction**
  - More complex (powerful?) instructions
  - More registers: inter-register operations are quicker
  - Fewer instructions per program
  - Longer instructions
- **Fewer addresses per instruction**
  - Less complex (powerful?) instructions
  - Shorter instructions: faster fetch and decode
  - More instructions per program
  - Accumulator machines are the extreme(极端): one address

Sources of Operands

- Number of operands
  - OP in acc = acc op in e.g. EDSAC (use the AC as an implicit destination)
  - OP inout, in2 out = in op in2 e.g. IBM 360
  - OP out, in1, in2 out = in1 op in2 e.g. MIPS

- Can ALU operands be in memory (register-memory)?
  - No e.g. MIPS (only register-register)
  - Yes e.g. x86 (can also be register-register)

- How many different formats and addressing modes?
  - A few e.g. MIPS (因为是reg-reg模式，不直接操作内存)
  - A lot e.g. x86

Memory addressing modes(内存寻址模式)

- Direct(直接寻址, 数1001) Load R4, 1001
- Register indirect (寄存器间接寻址, 地址R1) Load R4, (R1)
  - Use the content stored at R1 as address
- Memory indirect (内存间接寻址, 地址1001) Load R4, (1001)
  - Use value at mem[1001] as address
- Based (基址寻址, 地址20+R1) Load R4, 20(R1)
  - Use 20+(R1) as address
- Indexed(索引寻址, 地址R2[R3]) Load R4, (R2+R3)
- Auto inc/decrement(自动增长寻址, 地址R1,R1++) Load R4, (R1)+
  - use (R1) as address, and inc. or dec. R1

Your turn

Address	Value (hex)
R1	0001
R2	0002
R3	3010
1999	0012
2000	0013
2001	0015
2002	0014
3010	2001
3011	0034
3012	0100

What value would be stored in R4 in each of the following cases?

Load R4, R2	Direct
Load R4, (R3)	Register indirect
Load R4, 1(R3)	Based
Load R4, (R2+R3)	Indexed

Your turn: solution

- `Load R4, R2` register: R4 = 0002
- `Load R4, (R3)` register indirect: R4 = M[3010] = 2001
- `Load R4, 1(R3)` displaced: R4 = M[3010 + 1] = M[3011] = 0034
- `Load R4, (R2+R3)` indexed: R4 = M[0002 + 3010] = M[3012] = 0100
- Note: 2001 is the value at 3010 – the answer for (R3), not an address to follow again (that would be memory indirect)

What makes a good/bad ISA

- Ease of Implementation (for Architect/Designer)
  - Can it lead to simple hardware implementation
- Ease of Programming (for Programmer/Compiler)
  - Can the compiler use the ISA effectively?
- Future Compatibility
  - ISAs may last 30+ years
  - Special Features, Address range, etc. need to be thought out

RISC: reduced instruction set computer

- Simple operations
  - 2-input, 1-ouput
  - few alternatives for accomplishing the same thing
- Simple data movements
  - ALU ops are register to register; only load/store instructions access memory – needs more registers
  - Few addressing modes (typically register + displacement for loads/stores)
- Simple instruction encoding
  - All instructions encoded in the same number of bits
- Examples:
  - MIPS, ARM, RISC-V

X86: complex instruction set computer (CISC)

- Variable length ISA (1-16 bytes)
- ALU ops can be register-register and register-memory
- Multiple memory accessing modes

- Has been extended many times
  - E.g., x86-64 (released in 2000)

Design trade-offs（设计约定）

- Can decode CISC instructions into internal “micro-ISA” (in RISC)

  可以将CISC指令解码为内部“微ISA”

  - This takes a couple of extra cycles and a few hundred thousand transistors

  这需要几个额外的周期和几十万个晶体管

  - External ISA for compatibility, internal ISA can be tweaked each generation

  外部ISA用于兼容性，内部ISA可以在每一代中进行调整

- Energy efficiency can be significantly different

  能源效率可能存在显著差异

Where does Hack fit?（Hack计算机适合哪些）

- **Fixed-length instructions:** every instruction is 16 bits
- **Harvard:** separate instruction memory and data memory
- **Two registers:** D (data) and A (address or data)
- **Register–memory:** ALU operands are D and either A or the memory word M[A]
- **One addressing mode:** the memory address is always the value in A (register indirect)
- **Two instruction formats:** one loads a constant into A; the other computes, stores and/or jumps
- *Hack's assembly language is covered next semester; next lecture we build the ALU that executes the computations*

Summary

1. Reviewed the components of a computer and the role of each register
2. Followed the fetch–execute cycle and traced programs on a simple accumulator machine (MARIE)
3. Compared von Neumann and Harvard architectures; introduced our Hack computer
4. Defined an ISA; compared stack, accumulator and register architectures
5. Number of addresses, addressing modes, RISC vs CISC
6. Next: building the adders and the ALU